

Twitter Sentiment Analysis: Detecting Hate Speech

Sai Yashwant Gantasala (2201174) , Ubaid Abbas (2201216) , Rongali
Chetana Priya (2201169)

Dr. Kuntal Dey

November 6, 2025

Introduction: The Power of NLP

- **Natural Language Processing (NLP)** is a highly active and important research area in data science.
- One of the most common applications of NLP is **Sentiment Analysis**.
- This domain has reshaped how businesses operate, influencing everything from opinion polls to entire marketing strategies.
- **Why is it so powerful?**
 - **Speed:** It can process thousands of text documents for sentiment in seconds.
 - **Scale:** A task that would take a team of people hours to complete manually is done almost instantly.
 - **Versatility:** Beyond sentiment, it can extract topics, themes, and named entities.

The Problem Statement: Detecting Hate Speech

Main Objective: To detect hate speech in tweets.

Project Task Definition

For the sake of simplicity, we define a tweet as containing **hate speech** if it has a racist or sexist sentiment associated with it.

Our task is to classify these racist or sexist tweets from all other tweets.

- **Label '1':** The tweet is racist/sexist.
- **Label '0':** The tweet is *not* racist/sexist.
- **Evaluation Metric:** The F1-Score is used to measure model performance, which is critical for imbalanced datasets like this one.

Preprocessing & Cleaning: The "Why"

Without Cleaning

- Data is messy and inconsistent.
- Missing values, duplicates, and outliers distort insights.
- Irrelevant or noisy features confuse the model.
- Model learns patterns that don't generalize well.

With Cleaning

- Data is structured and reliable.
- Patterns become clearer and meaningful.
- Model training becomes faster and more stable.
- Predictions are accurate and interpretable.

Clean data → Clear insights → Better models.

- Data preprocessing ensures consistency, removes noise, and prepares data for learning.
- Skipping this step can lead to biased results and poor performance.

Preprocessing Cleaning: The "How"

The objective is to clean noise that is less relevant to finding the sentiment of tweets.

Our Cleaning Steps:

- ➊ **Remove User Handles:** Remove twitter handles (e.g., '@username') as they don't typically convey sentiment.
- ➋ **Remove Punctuation Special Characters:** Clean out symbols, numbers, and punctuation.
- ➌ **Tokenization:** Split tweets into individual words (tokens).
- ➍ **Remove Stopwords:** Remove common, low-value words (e.g., "the", "is", "a", "an").
- ➎ **Lemmatization/Stemming:** Reduce words to their root form (e.g., "running" → "run").

Goal

Good preprocessing leads to a **better quality feature space** for the models.

Exploration: Asking Questions

Exploring and visualizing data is an essential step to gain insights. Before plotting, we must think and ask questions.

Probable Questions:

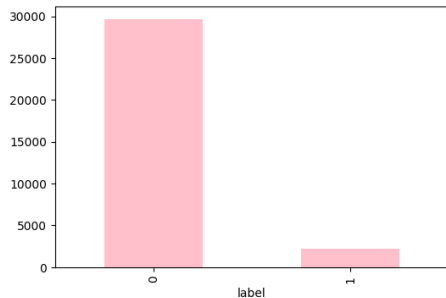
- What are the most common words in the entire dataset?
- What are the most common words for negative (label 1) and positive (label 0) tweets, respectively?
- How many hashtags are there in a tweet?
- What trends are associated with the data? Are they compatible with the sentiments?

Initial Data Check (from Notebook):

- Training data shape: (31962, 3)
- Test data shape: (17197, 2)
- No null values found in key columns.

Visualization: Data Characteristics

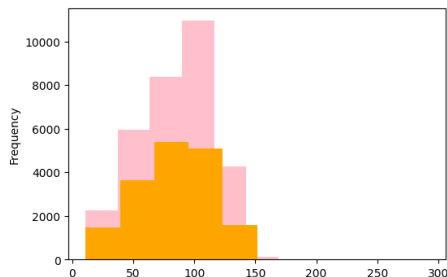
Label Distribution



Insight:

- The bar chart shows a severe class imbalance.
- There are far more non-hate tweets (0) than hate tweets (1).
- This is why F1-Score is a better metric than accuracy.

Tweet Length Distribution

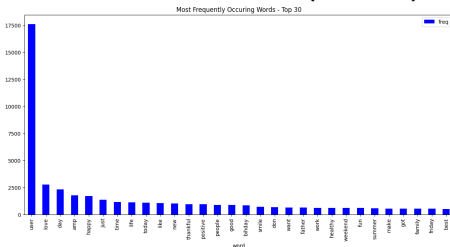


Insight:

- Non-hate (0) tweets: avg. 84 chars.
- Hate (1) tweets: avg. 90 chars.
- A slight tendency for hate tweets to be longer.

Visualization: Common Words

Most Frequent Words (Top 30)

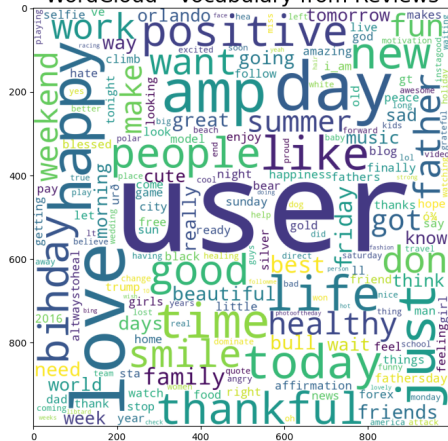


Insight:

- A bar chart showing the most common words after stopwords are removed.
- This helps identify common topics.

Word Cloud

WordCloud - Vocabulary from Reviews



Insight:

- A visual representation of the most frequent words.

Feature Extraction: Text to Numbers

Machine learning models don't understand words; they only understand numbers. Feature extraction is the process of converting our cleaned text into a numerical format.

Method 1: Bag-of-Words (BoW)

Creates a vector for each tweet where each entry is the *count* of a specific word. Simple and effective, but loses word order.

Method 2: TF-IDF (Term Frequency-Inverse Document Frequency)

An improvement over BoW. It *weights* words based on their importance.

- **Term Frequency (TF):** How often a word appears in a tweet.
- **Inverse Document Frequency (IDF):** Down-weights words that are common across all tweets (like "tweet" or "day").

This is the method used for the following models.

- Once we have our numerical TF-IDF features, we are ready to train our models.
- We split our data into a **training set** (for the model to learn from) and a **validation set** (to test its performance on unseen data).
- We will test four different classification algorithms to see which performs best on our specific task.
- **Models Tested:**
 - 1 Logistic Regression
 - 2 Support Vector Classifier (SVC)
 - 3 Decision Tree Classifier
 - 4 Random Forest Classifier

Model Results 1: Logistic Regression

Performance on TF-IDF Features

- **Training Accuracy:** 0.9991
- **Validation Accuracy:** 0.9312
- **F1 Score:** 0.536

Confusion Matrix

$$\begin{bmatrix} 7125 & 307 \\ 242 & 317 \end{bmatrix}$$

Model Results 2: Support Vector Classifier (SVC)

Performance on TF-IDF Features

- **Training Accuracy:** 0.9781
- **Validation Accuracy:** 0.9523
- **F1 Score:** 0.4993

Confusion Matrix

$$\begin{bmatrix} 7420 & 12 \\ 369 & 190 \end{bmatrix}$$

Model Results 3: [Decision Tree Classifier]

Performance on TF-IDF Features

- **Training Accuracy:** 0.9916
- **Validation Accuracy:** 0.9312
- **F1 Score:** 0.5359

Confusion Matrix

7125	307
242	317

Model Results 4: [Random Forest Classifier]

Performance on TF-IDF Features

- **Training Accuracy:** 0.9991
- **Validation Accuracy:** 0.9526
- **F1 Score:** 0.65

Confusion Matrix

7314	118
260	299

In this article, we learned how to approach a sentiment analysis problem from start to finish.

We followed a complete data science pipeline:

- ➊ **Problem Definition:** Clearly defined the goal and evaluation metric.
- ➋ **Preprocessing Cleaning:** Prepared the raw text for analysis.
- ➌ **Exploration Visualization:** Gained insights into the data's content and imbalances.
- ➍ **Feature Extraction:** Converted text into numerical features using TF-IDF.
- ➎ **Model Building Evaluation:** Trained and compared several classification models to find the best performer.

This journey demonstrates the critical importance of each step in an NLP task.

Thank You